```
str="learning python is very easy"
```

```
print(str[24:28],[])
```

```
easy
```

```
len(str)
```

```
28
```

```
i=len(str)
while i>0:
  print(str[i])
  i=i-1
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/tmp/ipykernel_2279/2625948527.py in <cell line: 0>()
      1 i=len(str)
      2 while i>0:
----> 3   print(str[i])
      4   i=i-1

IndexError: string index out of range
```

```
list1=[1,2,3,5,4,3,2,1,5,2]
for i in range(0,len(list1)):
  for j in range(i+1,len(list1)):
    if(list1[j]==list1[i]):
      del(list1[j])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/tmp/ipykernel_2279/1585151513.py in <cell line: 0>()
      2 for i in range(0,len(list1)):
      3   for j in range(i+1,len(list1)):
----> 4     if(list1[j]==list1[i]):
      5       del(list1[j])

IndexError: list index out of range
```

```
no=123
p=no%10
no=no//10
s=p*100
p=no%10
no=no//10
s=s+p*10
p=no%10
no=no/10
s=s+p
print(s)
```

```
321
```

```
class student:
  def show(self):
    print("i am in student class")
obj=student()
obj.show()
student().show() #anonymous object
```

```
i am in student class
i am in student class
```

```
class student:
  def __init__(self):
    print("i am a default constructor")
  def show(self):
    print("i am in student class")
obj=student()
obj.show()
```

```
i am a default constructor
i am in student class
```

```
class student:
  def __init__(self):
    print("i am a default constructor")
    print("hello")
  def show(self):
    print("i am in student class")
obj=student()
obj.show()
```

```
i am a default constructor
hello
i am in student class
```

```
class student:
  def __init__(self):
    self.name="fatema"
    self.age=22
    self.percentage=88

  def showinfo(self):
    print("name:",self.name)
    print("age:",self.age)
    print("percentage:",self.percentage)
s=student()
s.showinfo()
```

```
name: fatema
age: 22
percentage: 88
```

```
class student:
  def __init__(self):
    name="fatema"
    age=22
    percentage=88

  def showinfo(self):
    print("name:",name)
    print("age:",age)
    print("percentage:",percentage)
s=student()
s.showinfo()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_2279/1992009822.py in <cell line: 0>()
     10     print("percentage:",percentage)
     11 s=student()
---> 12 s.showinfo()

/tmp/ipykernel_2279/1992009822.py in showinfo(self)
      6
      7   def showinfo(self):
----> 8     print("name:",name)
      9     print("age:",age)
     10     print("percentage:",percentage)

NameError: name 'name' is not defined
```

```
#class with parameters
class student:
  def __init__(self,name,age,percent):
    self.name=name
    self.age=age
    self.percentage=percent

  def showinfo(self):
    print("name:",self.name)
    print("age:",self.age)
    print("percentage:",self.percentage)
s1=student("fatema",19,90)
s1.showinfo()
s2=student("anuj",19,100)
s2.showinfo()
```

```
name: fatema
age: 19
percentage: 90
name: anuj
age: 19
percentage: 100
```

```
#taking user input
class student:
  def __init__(self):
    self.name=input("name")
    self.age=int(input("age"))
    self.percentage=int(input("percent"))

  def showinfo(self):
    print("name:",self.name)
    print("age:",self.age)
    print("percentage:",self.percentage)
s1=student()
s1.showinfo()
s2=student()
s2.showinfo()
```

```
namefatema
age19
percent99
name: fatema
age: 19
percentage: 99
nameju
age3829
percent24
name: ju
age: 3829
percentage: 24
```

```
#taking user input
class student:
  def __init__(self,name,age,percentage):
    self.name=name
    self.age=age
    self.percentage=percentage

  def showinfo(self):
    print("name:",self.name)
    print("age:",self.age)
    print("percentage:",self.percentage)

name=input("enter name")
age=input("enter age")
percentage=("enter percent")
s1=student(name,age,percentage)
s1.showinfo()
```

Show hidden output

```
#python does not support function overloading, if such a case occurs, the last function is executed
class MyPython:
  def __init__(self):
    print("first constructor")
  def __init__(self):
    print("second constructor")

m=MyPython()
```

```
second constructor
```

```
#variable created using self is called instance variable
```

```
#delete instance variable
class mypython:
  def __init__(self):
    self.a=10
    self.b=20
    self.c=30
    self.d=40
```

```
  def func(self):
    del self.d
m=mypython()
print(m.__dict__)
m.func()
print(m.__dict__)
del m.c
print(m.__dict__)
```

```
{'a': 10, 'b': 20, 'c': 30, 'd': 40}
{'a': 10, 'b': 20, 'c': 30}
{'a': 10, 'b': 20}
```

```
#destructor
class mypython:
  def __init__(self):
    print("constructor is called")
  def __del__(self):
    print("destructor is called")
  def func(self):
    print("function code is executed")
m=mypython()
m.func()
```

```
constructor is called
function code is executed
```

```
#static variable
class pymython:
  x=20 #static variable
  def __init__(self):
    self.y=10 #instance variable

m1=pymython()
m2=pymython()
print("m1->",m1.x,m1.y)
print("m2->",m2.x,m2.y)
pymython.x=222
m1.y=111
print("m1->",m1.x,m1.y)
print("m2->",m2.x,m2.y)
```

```
m1-> 20 10
m2-> 20 10
m1-> 222 111
m2-> 222 10
```

```
#local variables
class test:
  def func1(self):
    a=100
    print(a)
  def func2(self):
    #a=11
    b=122
    print(b)
    print(a)

t=test()
t.func1()
t.func2()
```

```
100
122
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_2279/2398784753.py in <cell line: 0>()
     12 t=test()
     13 t.func1()
---> 14 t.func2()

/tmp/ipykernel_2279/2398784753.py in func2(self)
      8     b=122
      9     print(b)
---> 10     print(a)
     11
     12 t=test()

NameError: name 'a' is not defined
```

```
#local variables
class test:
  def func1(self):
    a=100
    print(a)
  def func2(self):
    a=11
    b=122
    print(b)
    print(a)

t=test()
t.func1()
t.func2()
```

```
100
122
11
```

Accept 5 paper marks, rollno, name. Calculate total and percentage. If percentage>=85,, student can take admission else can't take. Use CLASS, OBJECT, CONSTRUCTOR

```
class student:
  def __init__(self):
    self.name=input("enter name")
    self.roll=int(input("enter rollno"))

  def marks (mi,dmee,forr,itp,oe):
    mi=mi
    dmee=dmee
    forr=forr
    itp=itp
    rollno=rollno

  def accept():
   if(marks>=85):
      print("student can take admission")
    else:
      print("student cannot take admission")
```

```
#inheritance
class A:
  def showA(self):
    print("showA function")

class B(A):
  def showB(self):
    print("showB function")

obj=B()
obj.showA()
obj.showB()
```

```
showA function
showB function
```

```
#multi-level inheritance
class A:
  def showA(self):
    print("showA function")

class B(A):
  def showB(self):
    print("showB function")

class C(B):
  def showC(self):
    print("showC function")

obj=C()
obj.showA()
obj.showB()
obj.showC()
```

```
showA function
showB function
showC function
```

```
#multiple inheritance
class A:
  def showA(self):
    print("showA function")

class B():
  def showB(self):
    print("showB function")

class C(B,A):
  def showC(self):
    print("showC function")

obj=C()
obj.showC()
obj.showB()
obj.showA()
```

```
showC function
showB function
showA function
```

```
#hierarchal inheritance
class A:
  def showA(self):
    print("showA function")

class B(A):
  def showB(self):
    print("showB function")

class C(A):
  def showC(self):
    print("showC function")

obj=C()
obj.showC()
obj.showA()
```

```
showC function
showA function
```

```
#hybrid inheritance
class A:
  def showA(self):
    print("showA function")

class B(A):
  def showB(self):
    print("showB function")

class C():
  def showC(self):
```

```
    print("showC function")

class D(B,C):
  def showD(self):
    print("showD function")

obj=D()
obj.showC()
obj.showA()
obj.showD()
obj.showB()
```

```
showC function
showA function
showD function
showB function
```

```
class A:
  def showA(self):
    print("showA function")

class B(A):
  def showB(self):
    print("showB function")

class C():
  def showC(self):
    print("showC function")

class D(B,C):
  def showD(self):
    print("showD function")

obj=D()
obj.showC()
obj.showA()
obj.showD()
obj.showB()
```

```
showC function
showA function
showD function
showB function
```

```
class parent:
  def bike(self):
    print("splendor")

class child:
  pass

obj=child()
obj.bike()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_2279/3718194688.py in <cell line: 0>()
      7
      8 obj=child()
----> 9 obj.bike()

AttributeError: 'child' object has no attribute 'bike'
```

```
#function overriding
class parent:
  def bike(self):
    print("splendor")

class child:
  def bike(self):
    print("bullet")

obj=child()
obj.bike()
```

```
bullet
```